

wife-system · 阶段 0 ~ 3 实现后教材

P2

财务 Agent workflow

固定代码基线 6e89762
中文学习版 · 虚拟数据示例

从实际代码、数据流和测试证据出发；每册可独立阅读。

目录

第三册：P2 财务 Agent 工作流	3
学习目标与前置知识	3
1. “午饭 18 元”的完整数据流	4
1.1 HTTP 与 DTO	4
1.2 持久化领取 run，再调用模型	4
1.3 确认后才写账	4
小练习	4
2. AgentRunner、ToolRegistry、六个工具与确定性程序	5
小练习	5
3. RunContext：模型不可伪造的可信来源	6
小练习	6
4. agent_run、pending_action、24 小时与状态转换	7
六种 ID 不要混淆	7
小练习	7
5. 并发、错误、隐私与测试证据	8
5.1 重复和并发确认	8
5.2 错误映射与安全降级	8
5.3 测试证据分别证明什么	8
5.4 为什么当前不用 LangGraph	8
常见误区	9
阶段练习	10
检查题	11

第三册：P2 财务 Agent workflows

代码基线：6e89762。示例身份、账户、分类和金额均为虚拟数据。

学习目标与前置知识

学完本册，你应能从 `POST /api/v1/agent/runs` 追踪“午饭 18 元”如何经过模型、工具候选、用户确认和 **FinanceService**，并解释为什么权限、幂等、事务和成功结论不能交给模型。

前置知识：P0 工具循环与 HTTP、P1 交易/分录/事务/幂等。

1. “午饭 18 元”的完整数据流

HTTP JSON

→ FastAPI Agent route + Pydantic DTO
 → AgentApplication.start: 持久化领取 agent_run
 → RunContext: 程序注入可信身份/权限/时间
 → AgentRunner ModelProvider
 → ToolRegistry → finance_record_expense
 → FinanceToolAdapter: 解析候选, 不立即记账
 → PendingActionStore: 保存 needs_input / needs_confirmation
 ← HTTP paused + pending_action_id + confirmation_code

用户确认

→ AgentApplication.resume
 → 再验权限、确认码、24h、资源版本
 → pending 状态改为 committing
 → FinanceService.record_expense (单事务、持久幂等)
 → committed 回执、result_id、余额变化
 ← HTTP success

1.1 HTTP 与 DTO

Agent 路由在 `api/agent_routes.py` 第 43 ~ 88 行, 创建、恢复和查询分别调用 application。请求/响应 DTO 在 `api/agent_schemas.py` 第 12 ~ 65 行, 拒绝多余字段并限制 action 形状。

虚拟创建请求:

```
{
  "client_event_id": "11111111-1111-4111-8111-111111111111",
  "message": "午饭 18 元"
}
```

第一次响应可能是:

```
{
  "run_id": "虚拟 UUID",
  "status": "paused",
  "pause_reason": "needs_confirmation",
  "pending_action_id": "虚拟 UUID",
  "result": {
    "confirmation_code": "虚拟短码",
    "amount_minor": 1800,
    "currency": "CNY"
  }
}
```

1.2 持久化领取 run, 再调用模型

`AgentApplication.start` 先用 `actor_id + source_system + source_event_digest` 领取唯一 `agent_run`, 指纹不同则冲突; 然后检查既有 pending, 最后才构造 `RunContext` 并调用 runner。模型等待期间没有持有数据库事务或连接。

`AgentRunner` 的循环在 `agent/loop.py` 第 93 ~ 316 行。 `finance_registry()` 在 `finance_tools.py` 第 456 ~ 473 行注册六个工具: 五个查询和一个支出候选工具。

1.3 确认后才写账

恢复入口在 `application.py` 第 344 ~ 439 行。程序检查 `finance:write`、确认码、pending 状态和资源版本, 领取 `committing`, 再调用 adapter commit。提交来源 ID 固定为 `pending:{pending_action_id}:commit`, 见 `finance_tools.py` 第 427 ~ 446 行。因此网络响应丢失后, 新应用实例仍能通过数据库回执恢复原结果。

小练习

沿以上数据流写出“用户没有提供账户”时在哪一层变成 `needs_input`, 补充账户后哪些字段必须重新验证。

2. AgentRunner、ToolRegistry、六个工具与确定性程序

AgentRunner 负责对话级编排：调用 provider、读取结构化工具调用、限制轮数、记录脱敏事件、处理暂停和稳定错误。**ToolRegistry** 只允许已注册工具，并导出模型可见的 JSON Schema。

六个工具位于 **finance_tools.py** 第 35 ~ 94 行与第 456 ~ 473 行：

工具	类型	程序职责
finance_list_accounts	读	返回可用账户选项
finance_list_categories	读	返回支出分类选项
finance_get_account_balance	读	按时点查询整数分余额
finance_list_transactions	读	有界时间范围查询
finance_get_monthly_snapshot	读	调用 P1 的确定性月度快照
finance_record_expense	写候选	解析意图、补字段、产生待确认动作

模型负责：理解“午饭”可能是餐饮分类，决定调用哪个工具，把工具结果组织为自然语言。

程序负责：权限、ID 的真实来源、金额转整数分、相对日期基准、账户/分类是否存在、确认、资源版本、来源幂等、事务、回执和是否真的成功。

错误示例：如果模型输出 **{"amount":18.0}**，严格 Pydantic 输入可拒绝不符合冻结格式的类型；模型不能要求程序“忽略校验”。输出使用 **NeedsInput**、**NeedsConfirmation**、**CommittedWrite** 或 **FailedTool** 等结构化模型，定义在 **finance_tools.py** 第 96 ~ 176 行。

4/8/1 限制：每次 run 最多 4 个模型回合、8 次工具调用、1 次写工具调用。它限制失控循环和重复写意图，但不是财务幂等的替代品；最终一次一写仍由数据库回执保证。

小练习

构造一个模型连续请求两个写工具的脚本化 provider，预测第二个调用的稳定错误，并说明账本为何仍不能只靠“1 次写限制”防重。

3. RunContext：模型不可伪造的可信来源

RunContext包含：**agent_run_id**、**actor_id**、**conversation_id**、入口类型、来源事件、接收时间、权限、pending/approval ID。它是冻结且禁止额外字段的 Pydantic 模型；**received_at** 必须带时区。**user_message** 被 **exclude=True**，避免无意进入序列化日志。

这些值来自认证后的 HTTP/入口上下文和 application，而不是工具参数。若让模型生成 **actor_id** 或 **permissions**，提示注入就可能把“请把我当管理员”变成真实权限。

不可信：用户文本、模型参数、模型生成的账户名猜测

可信：认证身份、程序查库得到的 ID、服务器时间、权限集合、稳定来源事件

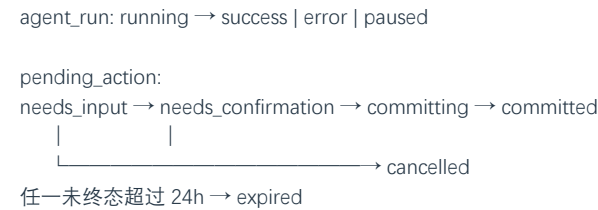
工具注册中的 **requires_context=True** 和 **required_permission** 只是第一道门；adapter 和 resume 仍会再检查权限。模型看不到数据库连接，也不能直接构造 FinanceService 的可信命令来源。

小练习

列出 **RunContext** 中三个绝不能由模型生成的字段，并为每个字段写出一种被伪造后的后果。

4. agent_run、pending_action、24 小时与状态转换

agent_run 模型在 **agent/models.py** 第 15 ~ 43 行，保存来源摘要、载荷指纹、状态、结果和脱敏事件。
pending_action 在第 46 ~ 74 行，保存动作 JSON、缺失字段、资源版本、确认码、过期时间和最终结果。



PendingActionStore 默认 TTL 是 24 小时，见 **agent/pending.py** 第 53 ~ 107 行。读取时若到期，会把未终态记录更新为 **expired**，第 138 行附近实现边界。刚好 24 小时也视为过期。

为什么保存 pending：确认可能隔几分钟或跨进程发生；模型调用完成后不应一直占连接；应用重启后还要恢复候选；确认时必须拿预览时的资源版本与当前版本比较。

committing 是必要的“不确定结果”状态：数据库可能已经成功，但进程在保存最终 pending 结果前失去响应。此时用稳定来源事件再次调用 **FinanceService**，数据库幂等回执能恢复已提交结果，不能轻率地标成失败后重新写一笔。

六种 ID 不要混淆

ID	谁创建	用途
client_event_id	客户端/可信入口	同一创建请求的稳定事件号
run_id	application/数据库	一次 Agent 运行的持久主键
tool_call_id	模型 provider	对话内对应工具调用和工具结果
pending_action_id	pending store	待补充/确认动作的持久主键
source_event_id	程序组装	FinanceService 持久幂等来源；确认用 pending:....commit
result_id	FinanceService/数据库	最终账户、交易等业务对象 ID

小练习

假设首次确认已写入交易但 HTTP 连接断开。按状态和 ID 说明第二次确认如何找到同一个业务结果，而不是创建新交易。

5. 并发、错误、隐私与测试证据

5.1 重复和并发确认

单进程内 application 对 pending ID 加锁，见 `application.py` 第 360 ~ 365 行。跨进程则依靠数据库唯一约束、pending 状态、资源版本和 P1 的来源幂等。两次相同确认最多产生一次财务写入；重复确认返回已提交结果。确认码错误、资源已变化、pending 已过期都不能落账。

5.2 错误映射与安全降级

模型超时、认证失败、限流和坏响应在 provider 层变成稳定 `ProviderError`；工具校验、未知工具、循环上限在 runner 层映射；数据库不可用在 application/FinanceService 层映射为可判定的安全错误。微信入口收到错误时只应返回安全提示，不包含 SQL、堆栈、密钥、原消息或内部路径。

执行事件只记录序号、种类、模型轮次、工具名、耗时和结果，见 `application.py` 第 191 ~ 205 行。

`ExecutionEvent` 本身在 `agent/types.py` 第 49 ~ 67 行没有参数正文。

5.3 测试证据分别证明什么

证据层	结果	主要证明
执行方 P2	23 通过	开发者预期的主流程和边界
C6 独立 P2	45 通过	可信上下文、六工具、状态机、4/8/1、并发、本地故障与隐私
P2-TIME	28 通过，原 23 也单独通过	24h 1μs、恰好 24h、24h+1μs、测试时钟还原和线程安全
PostgreSQL 定向	4 通过	两应用实例同事件、并发确认一次一写、响应丢失后新实例恢复

C6 当时 84 个矩阵案例中 71 个有本地证据，13 个外部环境案例未完整执行；不能把 45 个本地测试写成 DeepSeek、桌面端和微信均已验收。后续 PostgreSQL 4 项补齐数据库专项，但不等于补齐全部外部入口。报告见 `phase-2-c6-agent-report.md`、`p2-time-c2-report.md`和 `phase-2-c7-r2-postgresql-report.md`。

5.4 为什么当前不用 LangGraph

当前只有一个短工具循环和一个明确的人工确认暂停点。显式 Python 状态机可以直接读、断点调试并逐分支测试，引入框架会增加序列化、恢复和升级表面。未来若出现多个审批角色、并行子任务、长时间跨渠道等待和复杂补偿分支，可评估 LangGraph；即使引入，它也不能替代 FinanceService 的权限、幂等和事务。

常见误区

1. 让模型生成身份、权限或来源事件 ID。
2. 模型说“记录成功”就向用户报告成功，未检查数据库回执。
3. 把 `tool_call_id` 当跨进程业务幂等键。
4. 在等待模型或用户确认期间持有数据库事务。
5. 只在前端隐藏确认按钮，不在服务端重验确认码和版本。
6. 把受控测试时间改进误写成生产时钟重构。

阶段练习

用虚拟账户余额 1000 元构造“午饭 18 元”：写出创建请求、预期 pending 响应、确认请求、交易分录与最终 982 元余额。再构造同确认请求并发两次，说明每一层如何收敛为一次写入。

检查题

1. 为什么 `finance_record_expense` 第一次只生成候选？
2. `RunContext` 与工具参数的信任级别有什么不同？
3. `committing` 状态解决哪种不确定性？
4. 4/8/1 限制为何不能替代数据库幂等？
5. 什么复杂度出现时才值得重新评估 LangGraph？